Operating Systems and Parallel Computing

Lecture 13

Prof. Matteo Turilli

Quiz 1, Tuesday 3 November: What to Revise

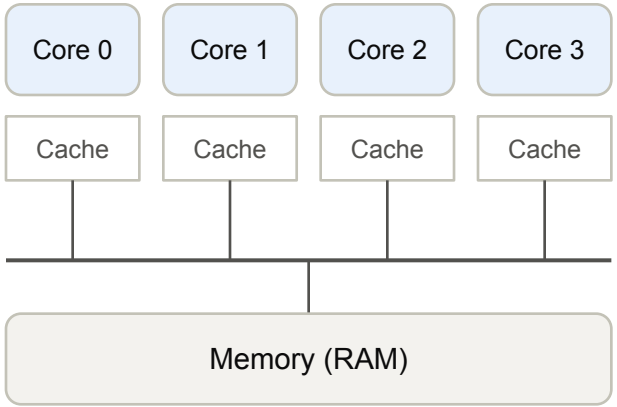
- **Lab 1 only**: the warm-up and all five tasks, including Tasks 4 and 5. For each one, explain the symptom, its cause and its fix.
- **Evidence**: read exit statuses, memory.events, the elapsed, user and system times of `/usr/bin/time`, `strace -c` counts, a cProfile listing and ps output.
- **Questions**: some show output like yours and ask you to interpret it; others give numbers and ask you to predict others.
- **Not on the quiz**: the Docker and GitHub setup, and command-line options to memorize.
- **Worth revisiting**: execute permission (Lectures 3 and 10), resident memory and the OOM killer (Lectures 6 and 7), where written bytes live (Lecture 9, slide 25), time sharing and more workers than CPUs (Exercises of Lectures 11 and 12, Exercise 3; Exercise 1 for one core).
- **Lab 1 explainers**: the four pages linked on Blackboard are good revision for the warm-up and Tasks 1–3.

Where We Are

- **Part I**: what the operating system does for our programs: processes, memory, files, permissions, scheduling.
- **Part II**: making our programs faster by spreading their work over many cores, many computers, or a GPU.
- In Lecture 12, processes competed for the cores. From now on, our own programs use many cores at once, on purpose.
- Today: what parallel computers look like, why NumPy is fast, when GPUs are fast, why moving data costs more than computing, and how big the “big machines” are.

Shared Memory or Distributed Memory

Shared memory

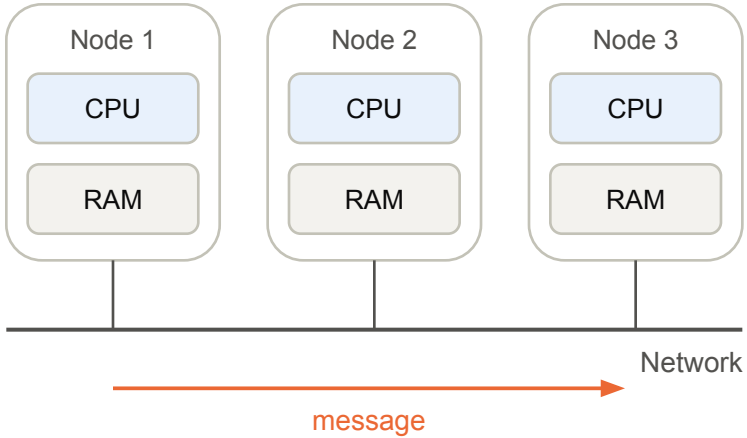


All cores read and write the same RAM.

Workers share data through memory.

Your laptop; one node of our cluster.

Distributed memory

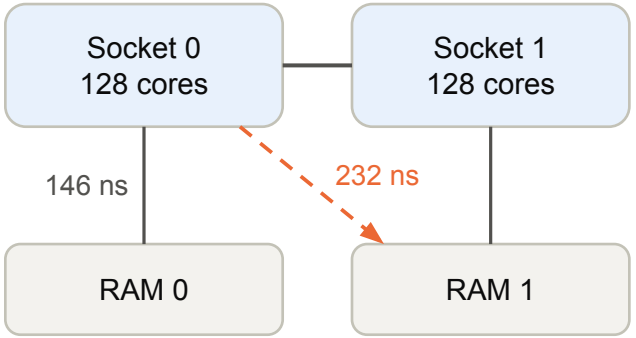


Each node has its own RAM and its own OS.

Data moves only as messages over the network. A cluster; most cloud services.

Shared Memory: One Address Space for All Cores

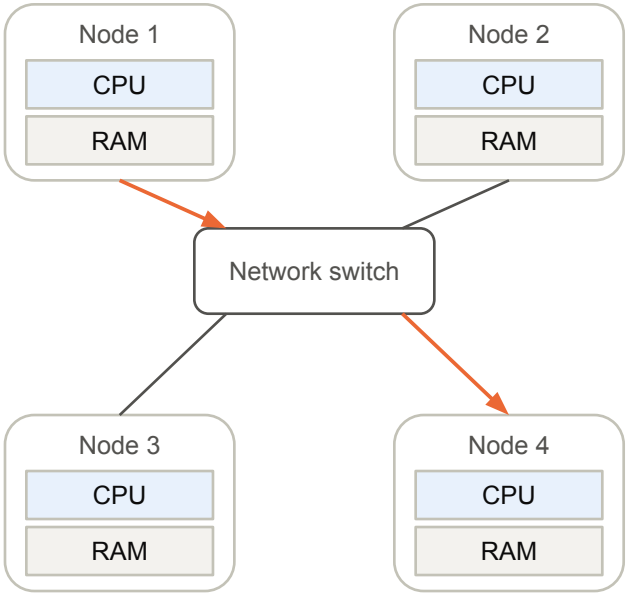
- **Shared memory**: every core can read and write every byte of RAM. Your laptop and our cluster nodes work this way.
- The threads of a process see the same variables, with no copies. We use this next session.
- The hardware keeps the caches coherent, but workers that update the same data still need locks (Lecture 5).
- All cores share the path to RAM: memory bandwidth limits how many cores can speed up a memory-bound program.
- With two sockets, each socket has its own RAM, and reaching the other socket's RAM takes longer: **NUMA**, non-uniform memory access.



Time to read one value from RAM on the cluster node called `haskell`: from the socket's own RAM, and from the other socket's RAM.

Distributed Memory: Many Computers, One Network

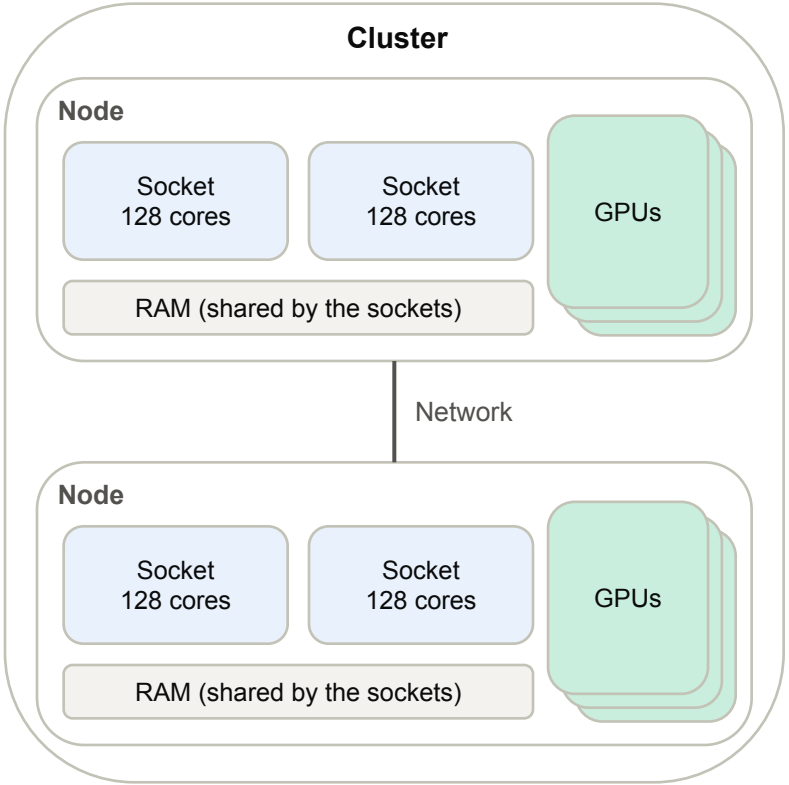
- **Why**: there is a physical limit to how much RAM, and how many CPUs and cores we can place on a single board and how fast they go work together.
- **Distributed memory**: each node is a complete computer with its own RAM and its own OS. No node can read another node's memory.
- To share data, a program sends **messages** over the network: copy the data, send it, receive it. Every message costs time.
- So the program splits its data, and each node works mostly on its own part.
- To get more cores and more memory, add nodes. The largest supercomputers have thousands.
- Dask (Session 24) works this way, and so do the MPI programs of scientific computing.



A message from node 1 to node 4 crosses the network.

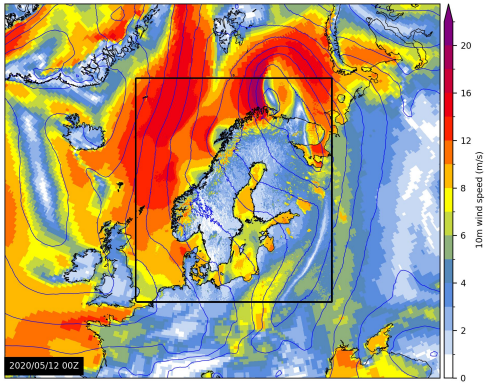
Clusters Are Both

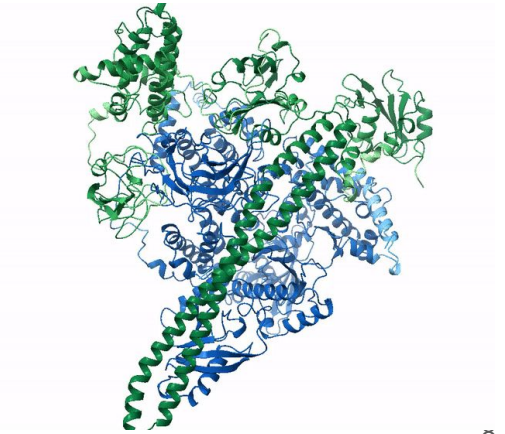
- A **cluster** is a set of shared-memory nodes connected by a fast network.
- Inside a node: shared memory. Between nodes: distributed memory.
- Parallel work happens at several levels: vector units inside each core, cores inside a node, nodes in the cluster, and GPUs attached to some nodes.
- A program can use one level or several. In this course: the cores of one node first, then many nodes with Dask, then a GPU.



Why We Build HPC Clusters

- **Some problems are too big for one computer**: their data does not fit in one node's memory, or their work would take months on one node.
- Weather forecasts, climate models, drug design, genomics, financial risk and the training of AI models all run on clusters of many nodes.
- Meta trained its Llama 3 language model (405 billion parameters) with 3.8 x 10^{25} operations, on 16,000 GPUs in 2,000 servers. One 4-CPU job on our cluster would need about 5 million years.
- **A cluster divides the work**: with N nodes, a job can finish up to N times sooner, but only if its work splits into parts that rarely need to talk to each other (Session 23).
- Your project does the same at a smaller scale: one workload, many cores, a measured speedup.





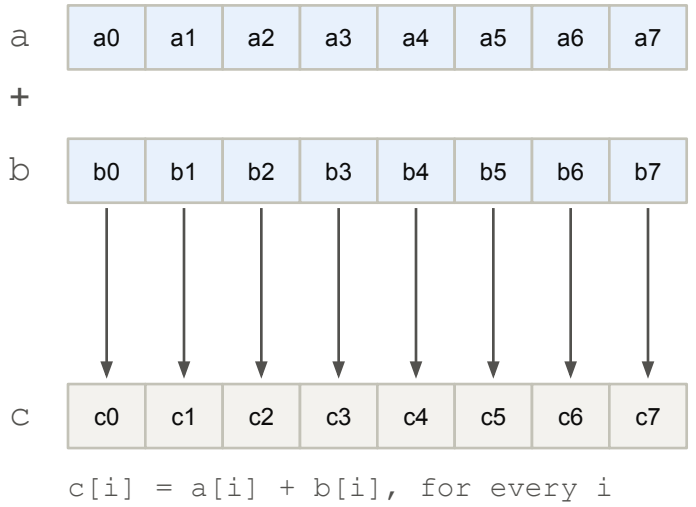
Flynn's Taxonomy

- Michael Flynn classified computers in 1966 by how many instruction streams and how many data streams they process at once.
- Real machines combine the classes: a cluster is MIMD, and every core inside it runs SIMD instructions.
- When NumPy adds two arrays, each instruction adds several numbers at once: up to 8 doubles on the processors of our cluster (AVX-512).

	One data stream	**Many data streams**
One instruction stream	**SISD**: one instruction on one value at a time. The classic single-core computer.	**SIMD**: one instruction on many values at once. Vector units in every core (NumPy uses them), GPUs.
Many instruction streams	**MISD**: rare; no common examples.	**MIMD**: independent cores, each running its own instructions on its own data. Multicore processors, clusters.

Element-wise Operations: a + b

- `a` and `b` are two arrays of the same length: here, a million numbers each.
- `c = a + b` computes every element on its own: c[0] = a[0] + b[0], c[1] = a[1] + b[1], and so on.
- `a * b` works the same way, with a multiplication. So does `df.price * df.qty` in pandas.
- For each result the computer reads two numbers, does one operation and writes one number.
- The next two slides: how Python does this a million times, and how NumPy does it.



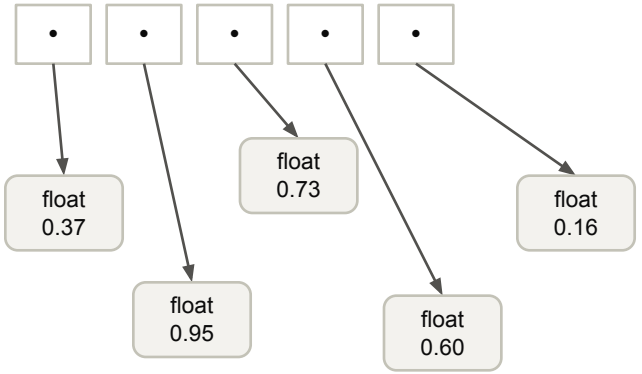
In Python: A Loop, One Number at a Time

- With lists `la` and `lb` (the same numbers), the loop is:

  ```
  c = [x + y for x, y in zip(la, lb)]
  ```

- Python first turns this line into bytecodes: small instructions such as “take the next pair”, “add”, “append”. The interpreter, a program written in C, runs them one by one.
- For every number, the Python interpreter:
  - reads the next bytecodes and works out what to do,
  - follows two pointers to the number objects,
  - checks that both are numbers it can add,
  - adds them,
  - creates a new object for the result.
- Only one of the five steps is the addition. In the container: 20.7 ms for a million numbers, about 20 ns each.

A Python list: pointers to number objects



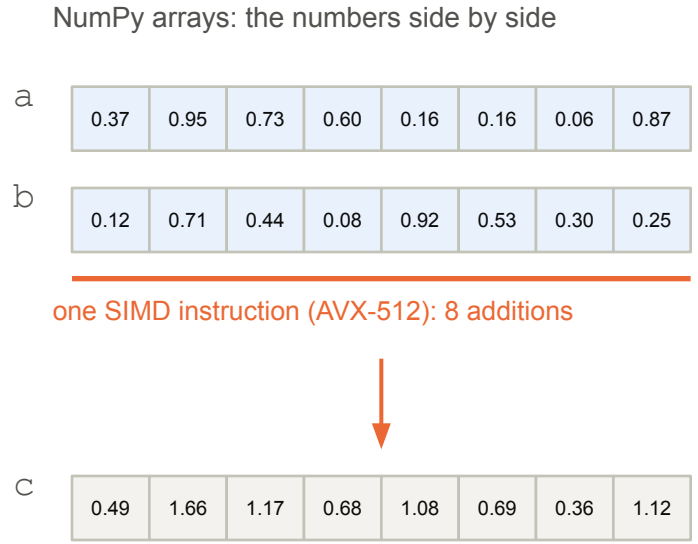
Each number lives in its own object, somewhere in memory.

In NumPy: One Compiled Loop, Then SIMD

- With NumPy arrays, the same work is one call:

  `c = a + b`

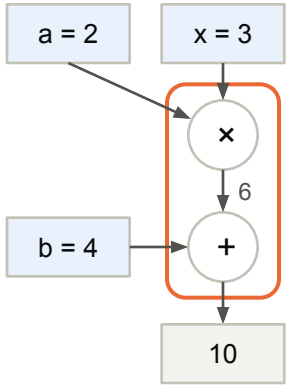
- NumPy runs one loop written in C: no bytecodes, no type checks, no new object for each number.
- The numbers sit side by side in memory, 8 bytes each, like the rows on the right.
- **SIMD**: because they sit side by side, one instruction loads 8 neighbors of `a` and 8 of `b` into wide registers and adds the 8 pairs at once (AVX-512 on our cluster; 2 pairs on Apple laptops, NEON).
- In the container: 0.18 ms for a million numbers, more than 100 times faster than the loop.



Multiply-Add: Two Operations, One Instruction

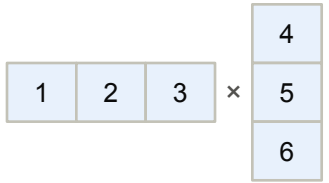
- A multiply-add computes `a * x + b`: one multiplication, then one addition.
- Processors do both with a single instruction, **FMA** (fused multiply-add): 2 × 3 + 4 = 10 in one step.
- **MAC** (multiply-accumulate): add each product to a running total, `s = s + a[i] * b[i]`.
- A matrix product is made of MACs: each result is a row times a column, multiplied pair by pair and added up.
- So Exercise 1 counted 2 x 2000³ operations for `a @ a`: 2000³ multiply-adds, two operations each.
- The next slide's test does 8 multiply-adds for every number (a polynomial).

One multiply-add: a * x + b



one FMA instruction:
multiply, then add

Accumulate: a row times a column




```
s = 0
s = 0 + 1*4 = 4
s = 4 + 2*5 = 14
s = 14 + 3*6 = 32
```

Each line is one multiply-add.
For a @ a in Exercise 1: 2,000 per result.

SIMD Pays Off When There Is Work per Number

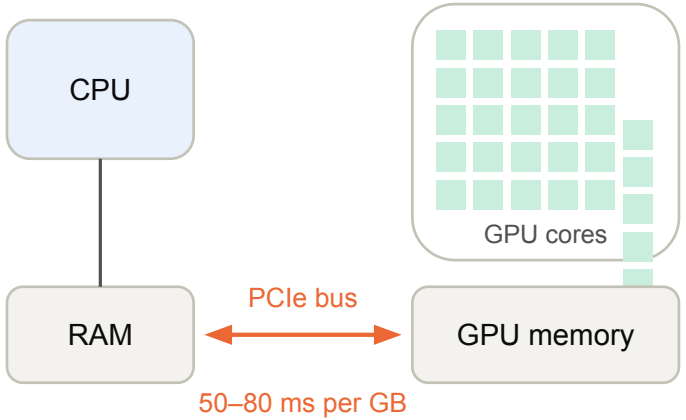
- `a + b`: SIMD is only 2 times faster. Each addition loads two numbers and stores one, and the loads and stores set the pace.
- 8 multiply-adds per number: 7 times faster. Now the arithmetic sets the pace, and 8 lanes work on 8 numbers at once. A matrix product (a @ a, Exercise 1) has this kind of work.
- From RAM, every version takes 0.8 ns per number: one core reads about 30 GB/s, and the lanes wait. Moving the data, not computing, is the limit.
- Interactive version: mturilli.github.io/ospc-materials/animations/vectorization.html

How the loop runs	a + b, in cache	8 multiply-adds, in cache	a + b, from RAM
One number per instruction	0.26 ns	0.99 ns	0.81 ns
SIMD, 4 per instruction (AVX2)	0.12 ns	0.25 ns	0.81 ns
SIMD, 8 per instruction (AVX-512)	0.12 ns	0.14 ns	0.80 ns

Time per number on IE cluster: the same C loop compiled three ways; 4,096 numbers (in cache) or 10 million (from RAM); best of 7 runs.

Accelerators: GPUs

- A **GPU** has thousands of simple cores that run the same instruction on different data, and has its own memory.
- The CPU copies the input to the GPU over the PCIe bus, and copies the results back. On our cluster: about 50 ms per GB to the GPU, 80 ms per GB back.
- A 4000 × 4000 matrix product in float32 takes the GPU 2 ms. With the copies in and out, it takes 16 ms.
- A GPU pays off when there is a lot of work for every byte copied: matrix products, deep learning. Session 26 uses CuPy.



- Our cluster has 5 GPUs: two NVIDIA RTX 6000 Ada (48 GB) on haskell, one RTX PRO 6000 Blackwell (96 GB) on julia, and two RTX PRO 6000 Blackwell (96 GB) on Fortran (testing).
- Interactive version: mturilli.github.io/ospc-materials/animations/gpu-offload.html

Computing Is Cheap, Moving Data Is Expensive

- One core of haskell multiplies matrices at about 100 billion operations per second: in the time of one read from RAM, it could do 15,000 of them.
- Fast programs keep their data close to the cores that use it: caches, batching, fewer copies.
- Interactive version: mturilli.github.io/ospc-materials/animations/compute-or-move.html

Where the data is	Time to get it	If 1 ns were 1 second
L1 cache, inside the core	1 ns	1 second
L2 cache	3.4 ns	3.4 seconds
L3 cache, shared by 8 cores	13 ns	13 seconds
RAM of the same socket	146 ns	2.4 minutes
RAM of the other socket	232 ns	3.9 minutes
SSD (`/scratch`), one 4 KB block	80 µs	22 hours
Another node, there and back (`ping`)	0.19 ms	2.2 days

How Big Is Exascale?

- **Exa =** 10^{18}. If all 8 billion people on Earth did one multiplication per second, they would need almost 9 years to match one second of LineShine.
- The top machine does the work of about 40,000 clusters like ours.

Machine (TOP500 rank)	Where	Cores	GPUs	Operations/s	Power
LineShine (1)	Shenzhen, China	13.8 million	none: CPUs only	2.2×10^{18}	42 MW
El Capitan (2)	Livermore, USA	11.3 million	45,000 AMD MI300A	1.8×10^{18}	30 MW
JUPITER Booster (5)	Jülich, Germany	4.8 million	23,536 NVIDIA GH200	1.0×10^{18}	16 MW
MareNostrum 5 ACC (16)	Barcelona, Spain	663,040	4,480 NVIDIA H100	1.8×10^{17}	4.2 MW
Our cluster	IE	512	3 NVIDIA RTX 6000	about 5×10^{13}	
A laptop (container)		4	1 built in (not used)	1.4×10^{11}	

Operations per second: float64, measured with the HPL benchmark. TOP500, June 2026. Our cluster: an estimate from our CPU measurements; laptop: Exercise 1.

Exascale and High Performance Computing in EU



Exascale Systems

JUPITER | Jülich Supercomputing Centre, Jülich
1 exaFLOP/s

Alice Recoque | CEA/GENCI, Bruyères-le-Châtel

Pre-Exascale Systems

LUMI | CSC, Kajaani
~0.54 exaFLOP/s

Leonardo | CINECA, Bologna
~0.32 exaFLOP/s

MareNostrum 5 | Barcelona Supercomputing Centre, Barcelona
~0.3 exaFLOP/s

Petascale Systems

MeluXina | LuxProvide, Bissen
~0.02 exaFLOP/s

Karolina | IT4Innovations National Supercomputing Center, Ostrava

Discoverer | Sofia Tech Park, Sofia

Vega | IZUM, Maribor

Deucalion | Univ. Minho, Riba de Ave

Curious Corner: A Supercomputer in a Chapel

- Since 2005, the Barcelona Supercomputing Center has kept its **MareNostrum** supercomputers in Torre Girona, a former chapel on the campus of the Polytechnic University of Catalonia.
- MareNostrum 3 (in the photos, 2015): 3,056 nodes with 48,896 cores, connected by an InfiniBand network, in a glass room built inside the chapel.
- The same design as our cluster, at a larger scale: every node is a shared-memory computer, and the nodes together form a distributed-memory machine.
- MareNostrum 4 (2017): 165,888 cores and 390 TB of RAM in its general-purpose part, for 11.15 petaflops (11 million billion operations per second) at 1.3 MW.
- MareNostrum 5, a European (EuroHPC) supercomputer, has been running since December 2023.





Which Parallelism Fits Your Project?

Your workload looks like	Try first	Session
A Python loop over array elements or table rows	Vectorize: NumPy, pandas columns	Today
Many independent runs: parameters, folds, files	Processes, joblib, or one job per run	21
Parallel libraries: NumPy, scikit-learn	Set threads and n_jobs; avoid nesting	22
Data larger than memory, or work for many nodes	Dask	24
Lots of math per byte, such as matrix products	A GPU with CuPy	26

- Whatever you choose, first time one run on one core: it is the baseline of every speedup you report. Session 23 measures speedup and efficiency on the cluster.
- Your proposal (due 4 November) should say which row your workload is in.

This is an interactive class

- Both exercises run in the course container on your laptop.
- Start a shell in the course container:

  `docker compose run --rm ospc`
- `python3 -m timeit -s SETUP STATEMENT` runs SETUP once, then runs STATEMENT many times and prints the best time per loop.
- **GFLOP/s**: billions of floating-point operations per second. **GB/s**: billions of bytes moved per second.
- Each exercise has three steps:
  - Make your prediction,
  - Run the commands,
  - Answer the questions.

Exercise 1: Compute or Move?

- `a+b` adds two arrays of 10 million numbers (80 MB each):
  - 10 million additions on 240 MB read and written.
- `a@a` multiplies a 2000 × 2000 matrix (32 MB) by itself:
  - $2 \times 2000^3 = 16$ billion operations.
- **Predict**: which one does more operations per second? How many times more?

```
S='import numpy as np; a = np.random.rand(10_000_000); b = np.random.rand(10_000_000)'
M='import numpy as np; a = np.random.rand(2000, 2000)'
python3 -m timeit -s "$S" 'a + b'
python3 -m timeit -s "$M" 'a @ a'
```

- **Predict**: now NumPy may use only one thread. How much slower will `a @ a` be?

```
OPENBLAS_NUM_THREADS=1 python3 -m timeit -s "$M" 'a @ a'
```

Exercise 1: Questions

1. How long did `a+b` and `a@a` take? How many GFLOP/s is each?
2. How many GB/s did `a+b` move?
3. Why does `a@a` do so many more operations per second than `a+b`?
4. How much slower was `a@a` with one thread? What does this tell you about NumPy?
5. In pandas, `df.price * df.qty` works like `a+b`. Would more cores or a GPU make it much faster?

Exercise 1: Answers

1. How long did `a+b` and `a@a` take? How many GFLOP/s is each?
2. How many GB/s did `a+b` move?
3. Why does `a@a` do so many more operations per second than `a+b`?
4. How much slower was `a@a` with one thread? What does this tell you about NumPy?
5. In pandas, `df.price * df.qty` works like `a+b`. Would more cores or a GPU make it much faster?

1. In the container on an Apple laptop: `a+b` 5.3 ms, so 10 million / 5.3 ms ≈ 2 GFLOP/s; `a@a` 116 ms, so 16 billion / 0.116 s ≈ 140 GFLOP/s, 70 times more. On a cluster node with 4 CPUs: 1 and 210 GFLOP/s.
2. 240 MB / 5.3 ms ≈ 45 GB/s. Every `a+b` also gets a new 80 MB array from the OS, which takes about half of the time.
3. `a+b` uses each number once, so the core mostly waits for RAM (memory-bound). `a@a` reuses each number 2000 times from the caches, with SIMD instructions, so the cores mostly compute (compute-bound).
4. 460 ms instead of 116 ms: 4 times slower. NumPy hands `a@a` to OpenBLAS, which already ran one thread per CPU. More in Session 22.
5. Not much: it is memory-bound. A GPU first needs the data copied over PCIe (50–80 ms per GB), which takes longer than the multiplication on the CPU.

Exercise 2: Loop or Vector?

- `a` and `b` hold a million numbers; `la` and `lb` are the same numbers as Python lists.
- **Predict**: how many times faster is NumPy than the Python loop?

```
S='import numpy as np, pandas as pd; a = np.random.rand(1_000_000); b = np.random.rand(1_000_000)'
L='la = a.tolist(); lb = b.tolist()'
python3 -m timeit -s "$S" -s "$L" '[x + y for x, y in zip(la, lb)]'
python3 -m timeit -s "$S" 'a + b'
```

- **Predict**: a table of 100,000 sales. How many times faster is the column product than `apply`?

```
D='df = pd.DataFrame({"price": a[:100_000], "qty": b[:100_000]})'
python3 -m timeit -s "$S" -s "$D" 'df.apply(lambda r: r.price * r.qty, axis=1)'
python3 -m timeit -s "$S" -s "$D" 'df.price * df.qty'
```

Exercise 2: Questions

1. How long did the loop and `a + b` take? How many times faster is NumPy?
2. If not on the additions, where does the loop spend its time?
3. How long did `apply` and the column product take? Why is `apply` so slow?
4. Your project code loops over the rows of a DataFrame. What do you try first?

Exercise 2: Answers

1. How long did the loop and `a + b` take? How many times faster is NumPy?
2. If not on the additions, where does the loop spend its time?
3. How long did `apply` and the column product take? Why is `apply` so slow?
4. Your project code loops over the rows of a DataFrame. What do you try first?

1. In the container on an Apple laptop: loop 20.7 ms, `a + b` 0.18 ms, more than 100 times faster. On a cluster node: 32 ms and 0.21 ms.
2. In the interpreter: for each number, Python decodes bytecodes, follows pointers to two number objects, checks their types and creates a new object for the sum. NumPy runs one compiled loop with SIMD instructions.
3. `apply` 0.43 s, the column product 0.04 ms: about 10,000 times faster. `apply` calls a Python function once per row and builds a row object for each call; the column product is one NumPy operation on two columns.
4. Vectorize first: replace the loop with column operations. Then add cores (next sessions): the loop spread over 64 cores would still take 0.3 ms, slower than one vectorized line.

Homework and Next Time

- **At home**: replay the interactive pages linked on the slides, and repeat Exercise 2 on a DataFrame of your own.
- Read Rauber and Rünger, chapter 2: Flynn's taxonomy and the memory organization of parallel computers.
- **Lab 1**: the at-home part is due Monday 2 November, 23:59.
- **Quiz 1**: Tuesday 3 November, in the first 15 minutes of the session.
- **Project proposal**: due Wednesday 4 November, 23:59. Use the parallelism map to place your workload.
- Next time: Quiz 1; your first login and first batch job on the cluster; threads, the GIL and processes.